

Contents of the lecture

- (1) “case” Statement**
- (2) Example-1: Behavior model of 2 to 1 multiplexer using “case” statement**
- (3) Test bench of 2 to 1 multiplexer**
- (4) Software Demonstration**



```
case (<expression>)  
  expr1: sequential_statement;  
  expr2: sequential_statement;  
  .  
  .  
  .  
  .  
  exprn: sequential_statement;  
  default: sequential_statement;  
endcase
```

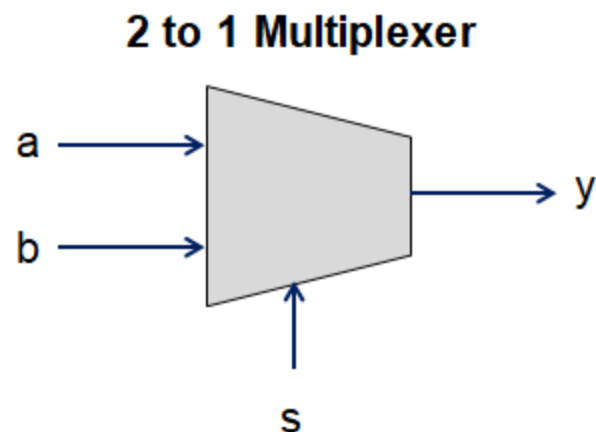
- **Each sequential statement can be a single statement or a group of statements within “begin.....end”**
- **Case statement can replace a complex “if.....else” statement**
- **The expression is compared to the alternatives (expr1, expr2, etc.) in the order they are written**
- **If none of the alternatives match, the default statement executed**

Example 1:

Write behavior model of 2 to 1 multiplexer using “case” statement

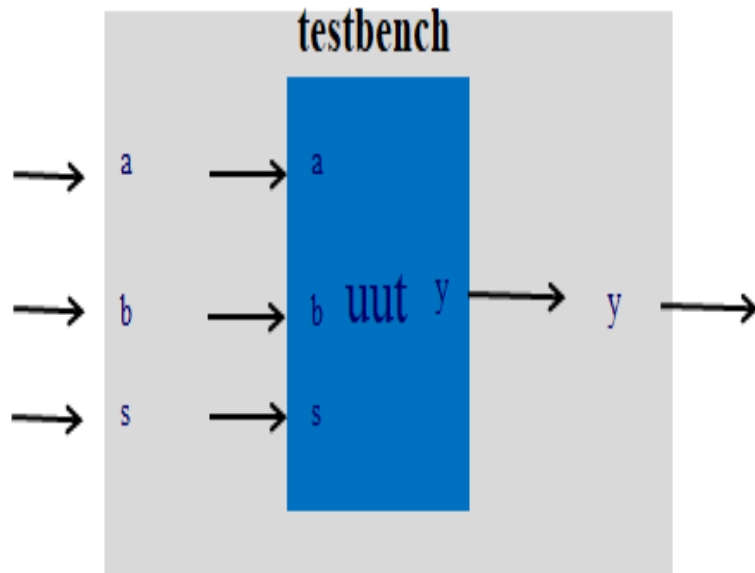
Solution:

```
module mux_2_to_1 (a,b,s,y);  
input a,b,s;  
output y;  
reg y;  
always @(a,b,s)  
case (s)  
1'b0:y=a;  
1'b1:y=b;  
default:y=0; // optional  
endcase  
endmodule
```



```
module mux_2_to_1 (a,b,s,y);
input a,b,s;
output y;
reg y;
always @(a,b,s)
case (s)
1'b0:y=a;
1'b1:y=b;
default:y=0; // optional
endcase
endmodule
```

```
module testbench;
reg a;reg b;reg s; wire y;
mux_2_to_1 uut (.a(a),.b(b),.s(s),.y(y));
initial
begin
$monitor($time,"a=%b,b=%b,s=%b,y=%b",
a,b,s,y);
#5 a=0;b=1;s=0;
#5 a=0;b=1;s=1;
#5 a=1;b=0;s=0;
#5 a=1;b=0;s=1;
#5 $finish;
end
endmodule
```



Software Demonstration

Contents of the lecture

- (1) Example-1: Behaviour model of 4 to 1 multiplexer using “case” statement
- (2) Test bench of 4 to 1 multiplexer
- (3) Software Demonstration



```
case (<expression>)  
  expr1: sequential_statement;  
  expr2: sequential_statement;  
  .  
  .  
  .  
  .  
  exprn: sequential_statement;  
  default: sequential_statement;  
endcase
```

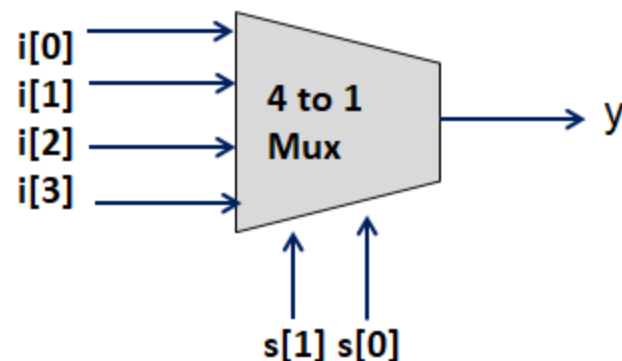
- **Each sequential statement can be a single statement or a group of statements within “begin.....end”**
- **Case statement can replace a complex “if.....else” statement**
- **The expression is compared to the alternatives (expr1, expr2, etc.) in the order they are written**
- **If none of the alternatives match, the default statement executed**

Example

Write behaviour model of 4 to 1 multiplexer using “case” statement

Solution:

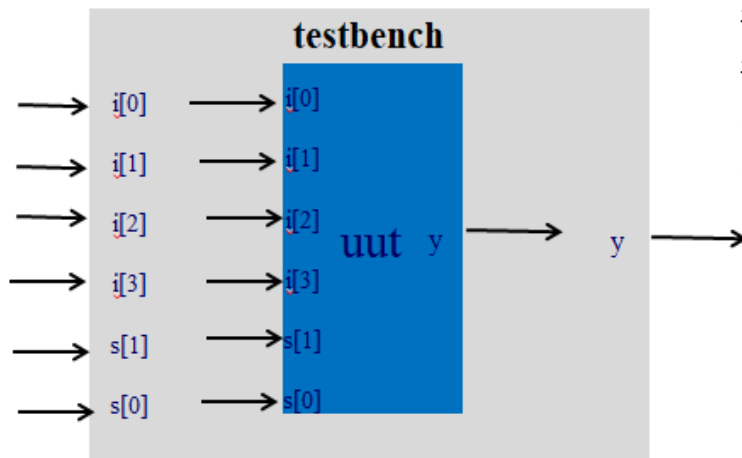
```
module multiplexer_4to1(i,s,y);
input [0:3]i;
input [1:0]s;
output y;
reg y;
always @(i,s)
case (s)
2'b00:y=i[0];
2'b01:y=i[1];
2'b10:y=i[2];
2'b11:y=i[3];
default:y=0;
endcase
endmodule
```



S[1]	s[0]	y
0	0	i[0]
0	1	i[1]
1	0	i[2]
1	1	i[3]


```
module mux4to1(i,s,y);
input [0:3]i;
input [1:0]s;
output y;
reg y;
always @(i,s)
case (s)
2'b00:y=i[0];
2'b01:y=i[1];
2'b10:y=i[2];
2'b11:y=i[3];
endcase
endmodule
```

```
module testbench;
reg [0:3]i;reg [1:0]s; wire y;
mux4to1 uut (.i(i),.s(s),.y(y));
initial
begin
$monitor($time,"i[0]=%b,i[1]=%b, i[2]=%b,i[3]=%b,
s[1]=%b,s[0]=%b,y=%b",i[0],i[1],i[2],i[3],s[1],s[0],y);
#5 i=4'b0101;s=2'b00;
#5 i=4'b0101;s=2'b01;
#5 i=4'b0101;s=2'b10;
#5 i=4'b0101;s=2'b11;
#5 i=4'b1100;s=2'b00;
#5 i=4'b1100;s=2'b01;
#5 i=4'b1100;s=2'b10;
#5 i=4'b1100;s=2'b11;
#5 $finish;
end
endmodule
```



Software Demonstration

Contents of the lecture

- (1) Write behavior model of 2 to 4 Decoder using “case” statement**
- (2) Test bench of 2 to 4 Decoder**
- (3) Software Demonstration**

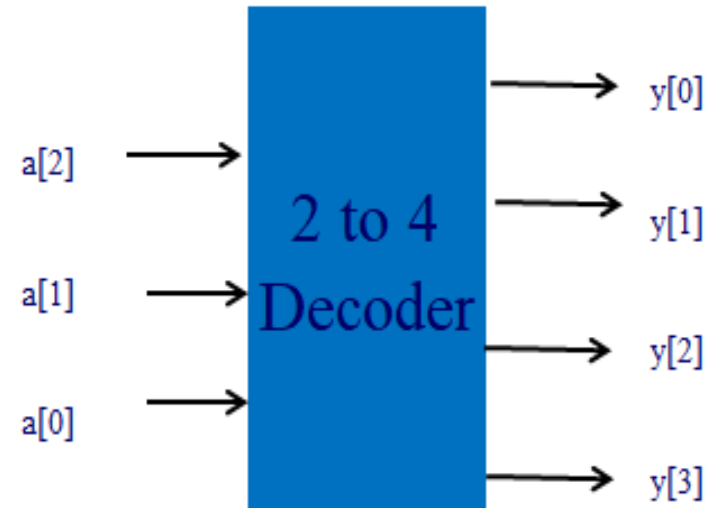


Dr. Yogesh Mishra
Professor of ECE< GMRIT

Example:

Write behavior model of 2 to 4 Decoder

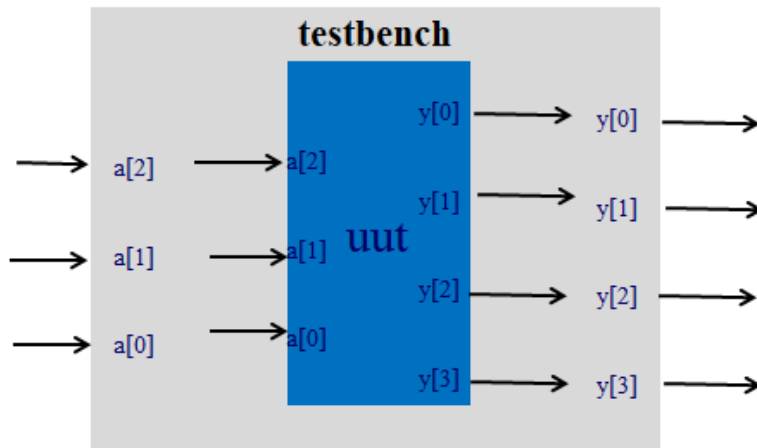
```
module decoder_2_to_4 (a,y);
input [2:0]a;
output [0:3]y;
reg [0:3]y;
always @(a)
case (a)
3'b100:y=4'b1000;
3'b101:y= 4'b0100;
3'b110:y=4'b0010;
3'b111:y=4'b0001;
default:y=4'b0000;
endcase
endmodule
```



a[2]	a[1]	a[0]	y[0]	y[1]	y[2]	y[3]
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

```
module decoder_2_to_4 (a,y);
input [2:0]a;
output [0:3]y;
reg [0:3]y;
always @(a)
case (a)
3'b100:y=4'b1000;
3'b101:y= 4'b0100;
3'b110:y=4'b0010;
3'b111:y=4'b0001;
default:y=4'b0000;
endcase
endmodule
```

```
module testbench;
reg [2:0]a; wire [0:3]y;
decoder_2_to_4 uut (.a(a),.y(y));
initial
begin
$monitor($time,"a[2]=%b,a[1]=%b,
a[0]=%b,y[0]=%b,y[1]=%b,y[2]=%b, ,y[3]=%b",
a[2],a[1], a[0],y[0],y[1],y[2],y[3]);
#5 a=3'b100;
#5 a=3'b101;
#5 a=3'b110;
#5 a=3'b111;
#5 a=3'b000;
#5 a=3'b101;
#5 $finish;
end
endmodule
```



Software Demonstration

Contents of the lecture

- (1) Write behavior model of one bit comparator using “case” statement**
- (2) Test bench of one bit comparator**
- (3) Software Demonstration**

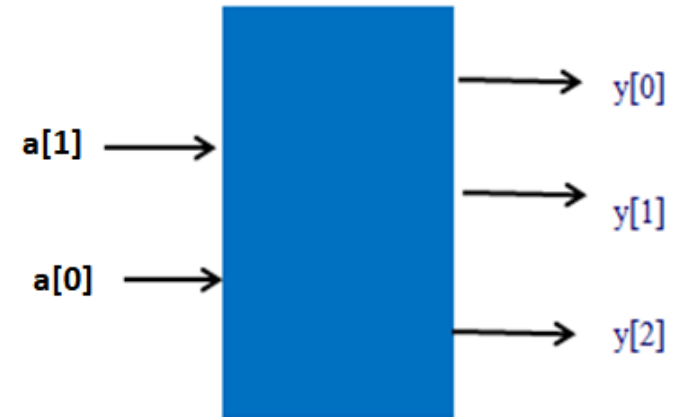


Dr. Yogesh Mishra
Professor of ECE< GARIT

Verilog code of One Bit Comparator using “if-else” Statement

```
module comparator_1_bit(a,y);  
input [1:0]a;  
output [0:2]y;  
reg [0:2]y;  
always @(a)  
case (a)  
2'b00:y=3'b100;  
2'b01:y=3'b010;  
2'b10:y=3'b001;  
2'b11:y=3'b100;  
default:y=3'b000;  
endcase  
endmodule
```

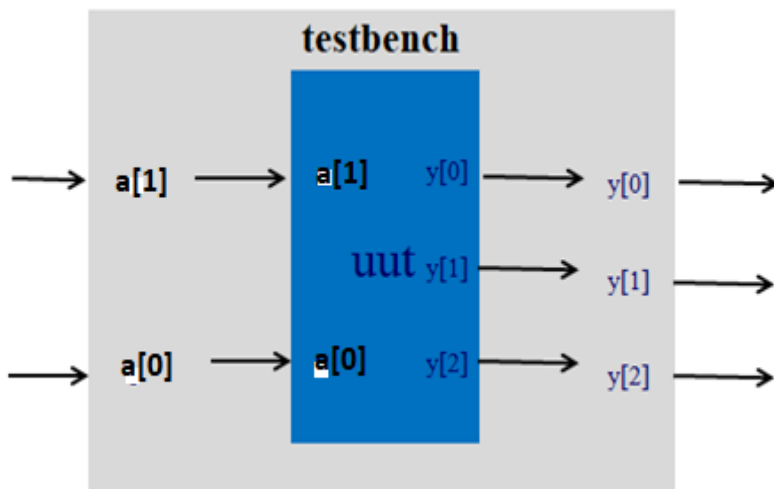
1-Bit Comparator



a[1]	b[0]	y[0]	y[1]	y[2]
0	0	1	0	0
0	1	0	1	0
1	0	0	0	1
1	1	1	0	0


```
module comparator_1_bit(a,y);
input [1:0]a;
output [0:2]y;
reg [0:2]y;
always @(a)
case (a)
2'b00:y=3'b100;
2'b01:y=3'b010;
2'b10:y=3'b001;
2'b11:y=3'b100;
default:y=3'b000;
endcase
endmodule
```

```
module testbench;
reg [1:0]a; wire [0:2]y;
comparator_1_bit uut (.a(a),.y(y));
initial
begin
$monitor($time,"a[1]=%b,a[0]=%b,y[0]=%b,y[1]=%b,y[2]=%b", a[1], a[0],y[0],y[1],y[2]);
#5 a=2'b00;
#5 a=2'b01;
#5 a=2'b10;
#5 a=2'b11;
#5 $finish;
end
endmodule
```



Software Demonstration of One Bit Comparator